

WEEK-9

Aim

To develop a user interface for a real-world scenario (a Blood Donation Camp) using React JSX, with a registration form that validates user input and displays the registered donors.

Description

This program is a single-page web application built with React. It lets people register as donors for a blood donation camp. The application is made of a single component, App, written in JSX (JavaScript XML), which allows HTML-like markup to be written inside JavaScript. A separate CSS file styles the page.

The registration form collects these details:

- Full name (text input)
- Age (number input)
- Mobile number (text input)
- Blood group (dropdown)
- Gender (radio buttons)
- Health confirmation (checkbox)

Concepts used:

- **Functional component:** the whole UI is a JavaScript function that returns JSX.
- **useState hook:** stores the form data, the error messages, and the list of donors.
- **Controlled inputs:** each field's value comes from state and is updated through the onChange event handler.
- **Event handling:** handleChange updates the form as the user types, and handleSubmit runs when the form is submitted.
- **Form validation:** validate() checks the following rules:
 - name has at least 3 characters
 - age is between 18 and 65
 - mobile number is a valid 10-digit number
 - blood group and gender are selected
 - the health checkbox is ticked

- **Conditional rendering:** error messages and the "no donors yet" text appear only when needed, using && and the ternary operator.
- **List rendering:** the .map() function displays each registered donor, with a unique key for each item.
- **Responsive design:** CSS Grid places the form and donor list side by side on large screens and stacks them on mobile.

Working: The user fills in the form and clicks Register. If any field is invalid, an error message appears below it. If all fields are valid, the donor is added to the "Registered donors" list and the form is cleared for the next entry.

Program Code

App.jsx:

```
import { useState } from "react";
import "./App.css";

// Scenario: Registration for a Blood Donation Camp
export default function App() {
  // 1. STATE: data that changes while the page is used
  const [form, setForm] = useState({
    name: "",
    age: "",
    phone: "",
    bloodGroup: "",
    gender: "",
    agree: false,
  });
  const [errors, setErrors] = useState({}); // error messages per field
  const [donors, setDonors] = useState([]); // list of registered donors

  // 2. HANDLE INPUT: runs every time the user types or selects something
  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setForm({ ...form, [name]: type === "checkbox" ? checked : value });
  }

  // 3. VALIDATION: check each field, return an object of error messages
```

```

function validate() {
  const newErrors = {};
  if (form.name.trim().length < 3) newErrors.name = "Enter your full name (min 3 letters).";
  if (form.age < 18 || form.age > 65) newErrors.age = "Donors must be between 18 and 65 years old.";
  if (!/^[6-9]\d{9}$/.test(form.phone)) newErrors.phone = "Enter a valid 10-digit mobile number.";
  if (!form.bloodGroup) newErrors.bloodGroup = "Select your blood group.";
  if (!form.gender) newErrors.gender = "Select your gender.";
  if (!form.agree) newErrors.agree = "Please confirm that you are in good health.";
  return newErrors;
}

// 4. SUBMIT: validate, then save the donor if everything is correct
function handleSubmit(e) {
  e.preventDefault(); // stop the page from reloading
  const newErrors = validate();
  setErrors(newErrors);
  if (Object.keys(newErrors).length === 0) {
    setDonors([...donors, form]); // add the new donor to the list
    setForm({ name: "", age: "", phone: "", bloodGroup: "", gender: "", agree: false }); // clear form
  }
}

// 5. UI: what appears on the screen
return (
  <div className="page">
    <header>
      <h1>Blood Donation Camp</h1>
      <p>Register to donate on Sunday, 10 AM – 4 PM.</p>
    </header>
    <main>
      {/* ----- REGISTRATION FORM ----- */}
      <form onSubmit={handleSubmit} noValidate>
        <h2>Donor registration</h2>
        <label>

```

Full name

```
<input name="name" value={form.name} onChange={handleChange} />
{errors.name && <span className="error">{errors.name}</span>}
</label>
<label>
```

Age

```
<input name="age" type="number" value={form.age} onChange={handleChange} />
{errors.age && <span className="error">{errors.age}</span>}
</label>
<label>
```

Mobile number

```
<input name="phone" value={form.phone} onChange={handleChange} maxLength={10} />
{errors.phone && <span className="error">{errors.phone}</span>}
</label>
<label>
```

Blood group

```
<select name="bloodGroup" value={form.bloodGroup} onChange={handleChange}>
  <option value="">Choose one</option>
  {[ "A+", "A-", "B+", "B-", "AB+", "AB-", "O+", "O-"].map((g) => (
    <option key={g} value={g}>{g}</option>
  ))}
</select>
{errors.bloodGroup && <span className="error">{errors.bloodGroup}</span>}
</label>
```

<fieldset>

<legend>Gender</legend>

```
{[ "Male", "Female", "Other"].map((g) => (
  <label key={g} className="inline">
    <input
      type="radio"
      name="gender"
      value={g}
```

```

        checked={form.gender === g}
        onChange={handleChange}
      />
    {g}
  </label>
))}
{errors.gender && <span className="error">{errors.gender}</span>}
</fieldset>
<label className="inline">
  <input type="checkbox" name="agree" checked={form.agree} onChange={handleChange} />
  I confirm that I am in good health.
</label>
{errors.agree && <span className="error">{errors.agree}</span>}
<button type="submit">Register</button>
</form>
{ /* ----- REGISTERED DONORS LIST ----- */}
<section>
  <h2>Registered donors ({donors.length})</h2>
  {donors.length === 0 ? (
    <p className="empty">No one has registered yet. Be the first!</p>
  ) : (
    <ul>
      {donors.map((d, index) => (
        <li key={index}>
          <span className="group">{d.bloodGroup}</span>
          <div>
            <strong>{d.name}</strong>
            <small>{d.age} yrs, {d.gender}, {d.phone}</small>
          </div>
        </li>
      ))}
    </ul>
  )}

```

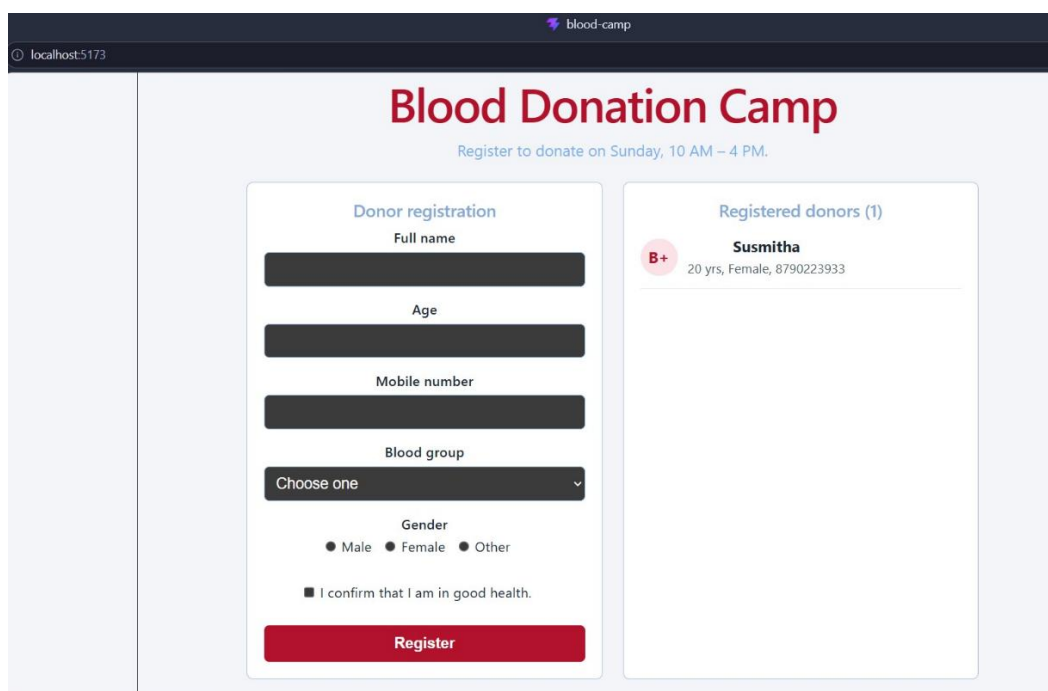
```

    )}
  </section>
</main>
</div>
);
}

```

Result

The registration form was successfully developed using React JSX. It validates the user's input and displays the registered donors on the page.



The screenshot shows a web application titled "Blood Donation Camp" with a subtitle "Register to donate on Sunday, 10 AM – 4 PM." The application is running on a browser at localhost:5173.

Donor registration

Full name

Age

Mobile number

Blood group
 Choose one ▼

Gender
☐ Male ☐ Female ☐ Other

☐ I confirm that I am in good health.

Register

Registered donors (1)

B+ **Susmitha**
 20 yrs, Female, 8790223933